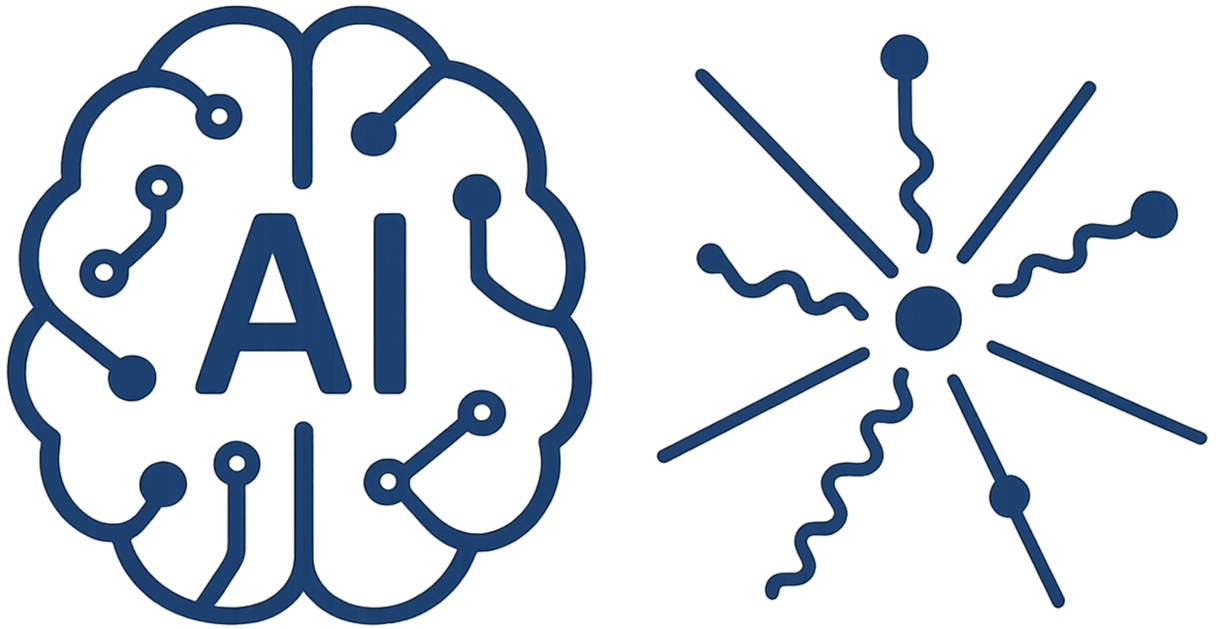
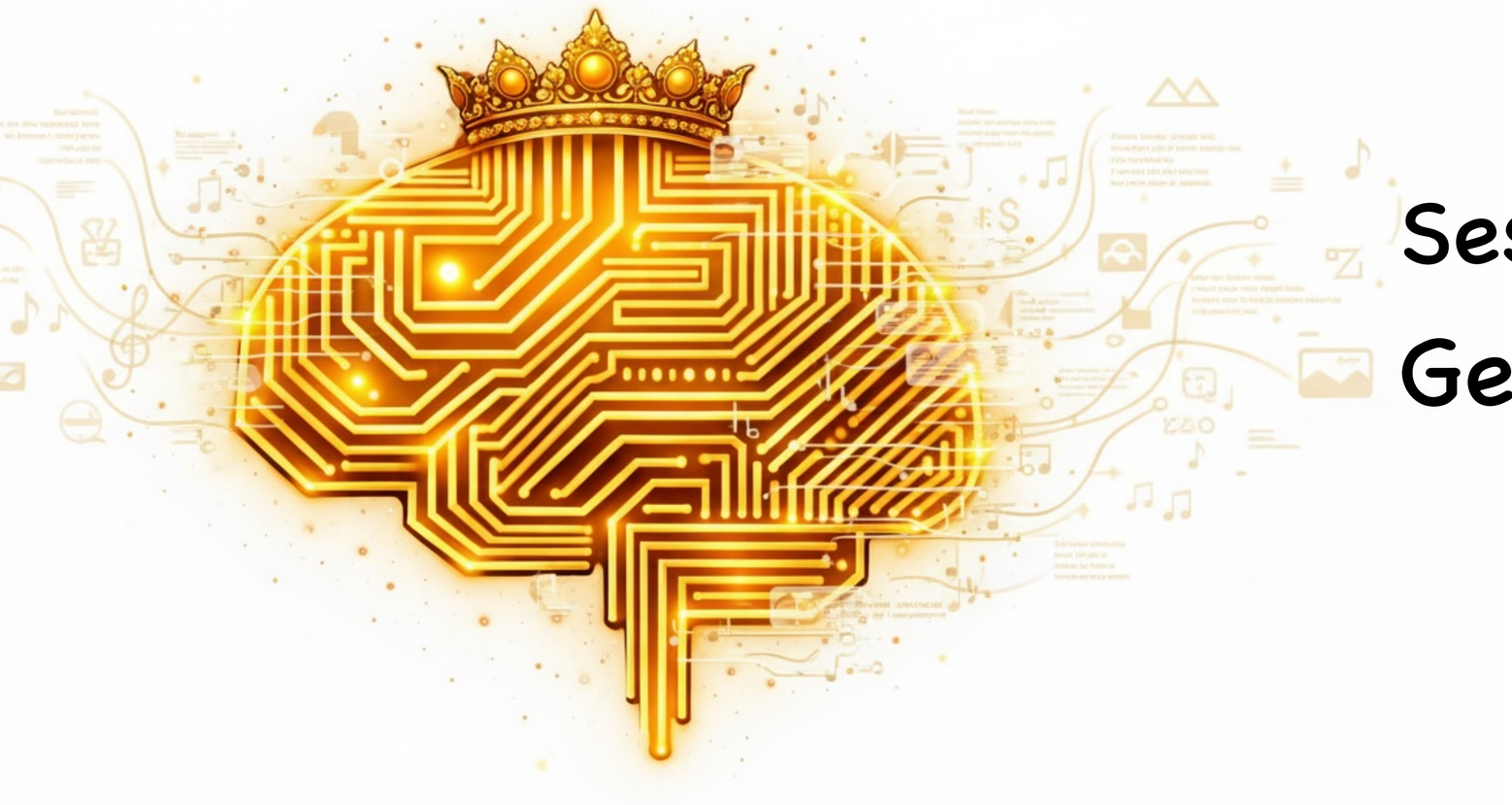
Introduction to

AI-Driven HEP

S. A. Fard - School of Physics (IPM)

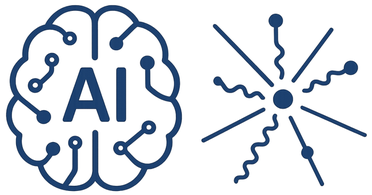


AI-Driven HEP 23



Session 23

Generative AI II



AI-Driven HEP 23

Reference

[1] Bishop, Christopher M., and Hugh Bishop. Deep learning: Foundations and concepts. Springer Nature, 2023.

[2] Simon J.D. Prince, Understanding Deep Learning, The MIT Press (2025), https://udlbook.github.io/udlbook/

[3] Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks, https://arxiv.org/abs/1703.10593

[4] Normalizing Flows: An Introduction and Review of Current Methods, https://arxiv.org/abs/1908.09257

[5] Normalizing Flows for Probabilistic Modeling and Inference, https://arxiv.org/pdf/1912.02762v2,

[6] Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks, https://arxiv.org/abs/1511.06434v2

https://generated.photos/

https://physics.ipm.ac.ir/~ansarifard/

Nonlinear Latent Models

$$p(x\,|\,z) \sim \mathcal{N}(x\,|\,Wz + \mu,\ \sigma^2 I) \qquad \rightarrow \qquad p(x\,|\,z, w) \sim \mathcal{N}(x\,|\,g(z, w),\ \sigma^2 I)$$

Nonlinear Latent Models

$$p(x\,|\,z) \sim \mathcal{N}(x\,|\,Wz+\mu,\ \sigma^2 I) \quad \rightarrow \quad p(x\,|\,z,w) \sim \mathcal{N}(x\,|\,g(z,w),\ \sigma^2 I)$$

Generator

$g \in \mathbb{R}^D$

Output of ML network

Latent vector

$z \in \mathbb{R}^K$

Input of ML network

Nonlinear Latent Models

$$p(x\,|\,z) \sim \mathcal{N}(x\,|\,Wz+\mu,\ \sigma^2 I) \quad \rightarrow \quad p(x\,|\,z,w) \sim \mathcal{N}(x\,|\,g(z,w),\ \sigma^2 I)$$

Generator

$g \in \mathbb{R}^D$

Output of ML network

Latent vector

$z \in \mathbb{R}^K$

Input of ML network

Generative path

1) draw sample from p(z)
2) input the sample to network
3) produce sample with mean g and covariance σ^2

Nonlinear Latent Models

$$p(x\,|\,z) \sim \mathcal{N}(x\,|\,Wz+\mu,\ \sigma^2 I) \quad \rightarrow \quad p(x\,|\,z,w) \sim \mathcal{N}(x\,|\,g(z,w),\ \sigma^2 I)$$

Generator

$g \in \mathbb{R}^D$

Output of ML network

Latent vector

$z \in \mathbb{R}^K$

Input of ML network

Likelihood path

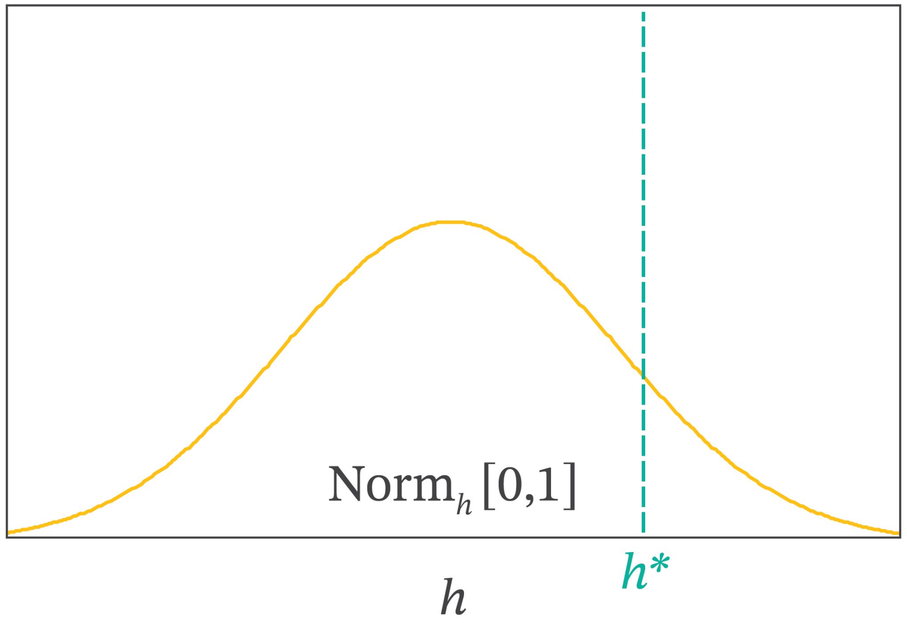
$$p(x\,|\,w) = \int \mathcal{N}(x\,|\,g(z,w),\ \sigma^2 \mathrm{I})\mathcal{N}(\mathrm{z}\,|\,0,\mathrm{I})\mathrm{dz}$$

the integral is analytically intractable

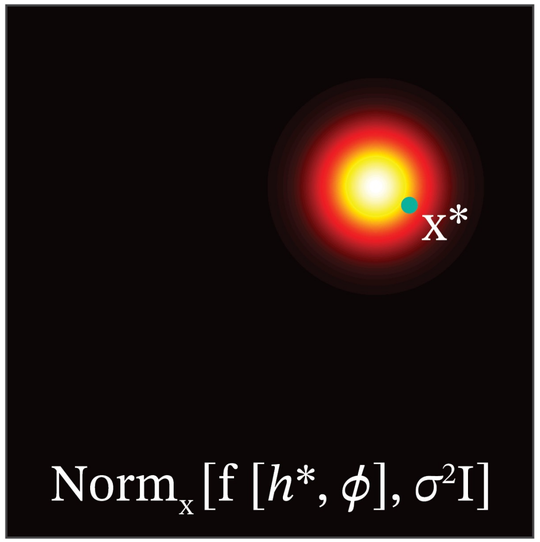
due to the highly nonlinear function g

Nonlinear Latent Models

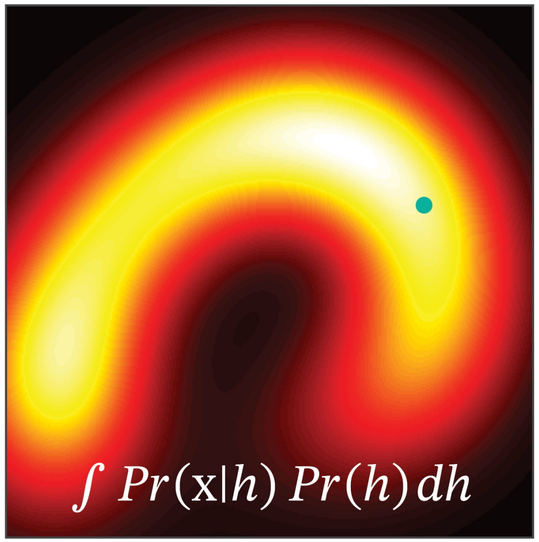
$Pr(h)$



$Pr(\mathrm{x}|h = h^*, \phi)$



$Pr(\mathrm{x})$



Adversarial Training

Goal is finding the distribution of data space

$$p(x \mid z, w) \sim \mathcal{N}(x \mid g(z, w),\ \sigma^2 I)$$

$$p(z) \sim \mathcal{N}(x \mid 0,\ I)$$ Mapping random noise to the data space

Adversarial Training

Goal is finding the distribution of data space

$$p(x \mid z, w) \sim \mathcal{N}(x \mid g(z, w),\ \sigma^2 I)$$

$p(z) \sim \mathcal{N}(x \mid 0,\ I)$ Mapping random noise to the data space

Introduce a second discriminator network

trained jointly with the generator network

distinguished between real examples from the data set and synthetic produced by the generator network

Loss function

Consider t as a binary target variable

t = 1 : real data

t = 0 : synthetic data

$d(x, \phi)$ probability that a data vector x is real

Discriminator network has a single output unit with a logistic-sigmoid activation function

Loss function

Consider t as a binary target variable

t = 1 : real data

t = 0 : synthetic data

$d(x, \phi)$ probability that a data vector x is real

Discriminator network has a single output unit with a logistic-sigmoid activation function

$$\mathscr{L}_D(\phi) = -\mathbb{E}_{x\sim p_{\text{data}}}[\log d(x;\phi)] - \mathbb{E}_{z\sim p(z)}[\log(1 - d(g(z;w);\phi))]$$

error is minimized with respect to ϕ but
maximized with respect to w

Training

Train Discriminator (Frozen Generator)

$$\{x_i\}_{i=1}^{m} \sim p_{\text{data}}$$

$$\{z_i\}_{i=1}^{m} \sim p(z) \qquad \tilde{x}_i = g(z_i; w)$$

$$\mathscr{L}_D = -\frac{1}{m}\sum_i \left[\log d(x_i;\phi) + \log(1 - d(\tilde{x}_i;\phi))\right]$$

Training

Train Discriminator (Frozen Generator)

$$\{x_i\}_{i=1}^{m} \sim p_{\text{data}}$$

$$\{z_i\}_{i=1}^{m} \sim p(z) \qquad \tilde{x}_i = g(z_i; w)$$

$$\mathscr{L}_D = -\frac{1}{m}\sum_i \left[\log d(x_i;\phi) + \log(1 - d(\tilde{x}_i;\phi))\right]$$

Train Generator (Frozen Discriminator)

$$z_i \sim p(z)$$

$$\tilde{x}_i = g(z_i; w)$$

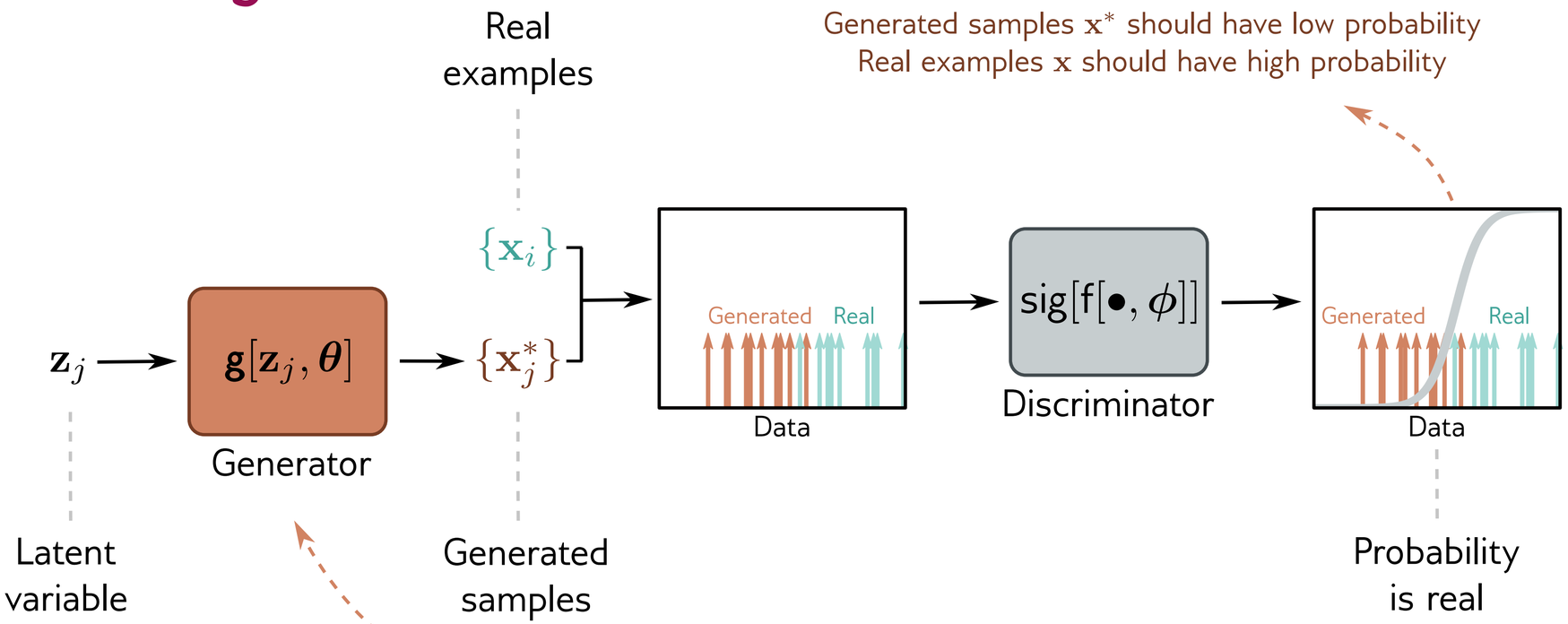
$$\mathscr{L}_G = -\frac{1}{m}\sum_i \log d(\tilde{x}_i;\phi)$$

Training

Discriminator loss, $L[\phi]$

$$-\sum_j \log\left[1-\text{sig}[\text{f}[\mathbf{x}_j^*,\boldsymbol{\phi}]]\right] - \sum_i \log\left[\text{sig}[\text{f}[\mathbf{x}_i,\boldsymbol{\phi}]]\right]$$

Generated samples $\mathbf{x}^*$ should have low probability
Real examples $\mathbf{x}$ should have high probability



Generator loss, $L[\boldsymbol{\theta}]$

$$\sum_j \log\left[1-\text{sig}[\text{f}[\mathbf{g}[\mathbf{z}_j,\boldsymbol{\theta}],\boldsymbol{\phi}]]\right]$$

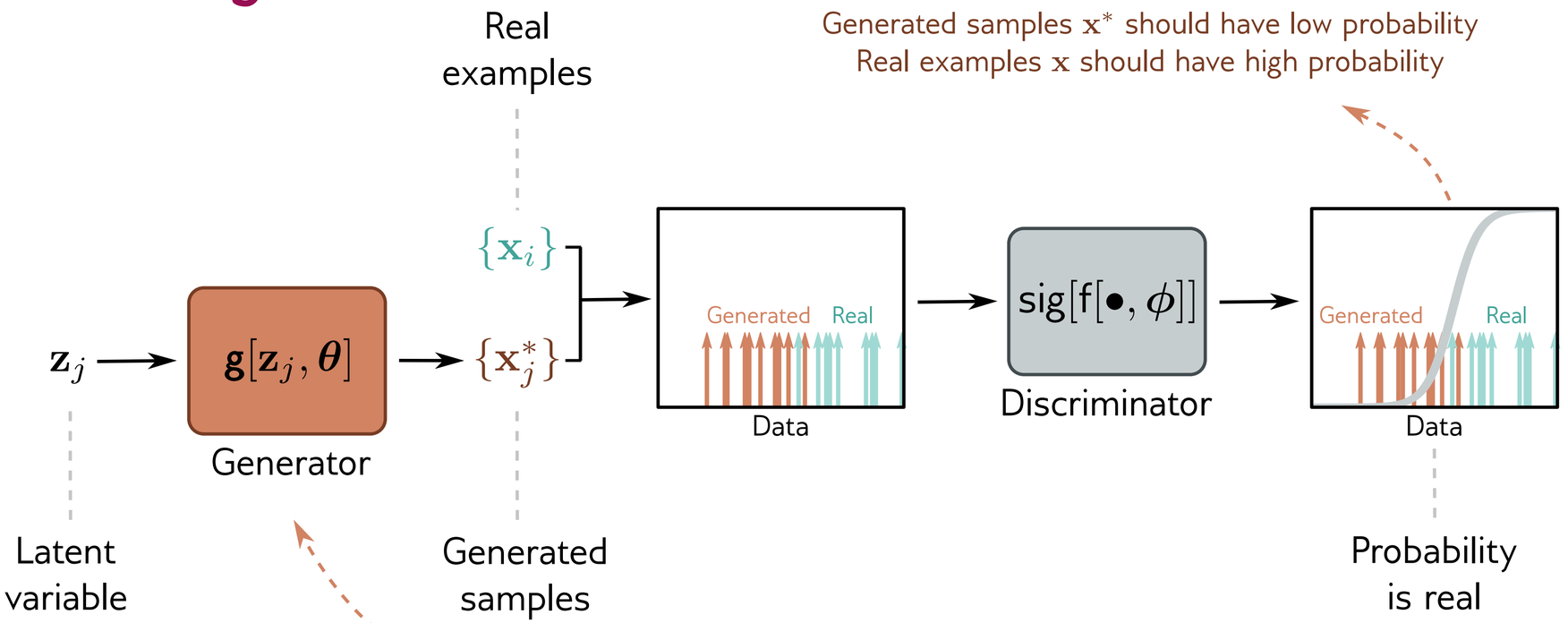
Generated samples $\mathbf{x}^* = \mathbf{g}[\mathbf{z},\boldsymbol{\theta}]$
should be assigned high
probability by discriminator

Training

Discriminator loss, $L[\phi]$

$$-\sum_j \log\left[1-\text{sig}[\text{f}[\mathbf{x}_j^*,\phi]]\right]-\sum_i \log\left[\text{sig}[\text{f}[\mathbf{x}_i,\phi]]\right]$$

Generated samples $\mathbf{x}^*$ should have low probability
Real examples $\mathbf{x}$ should have high probability



Generator loss, $L[\boldsymbol{\theta}]$

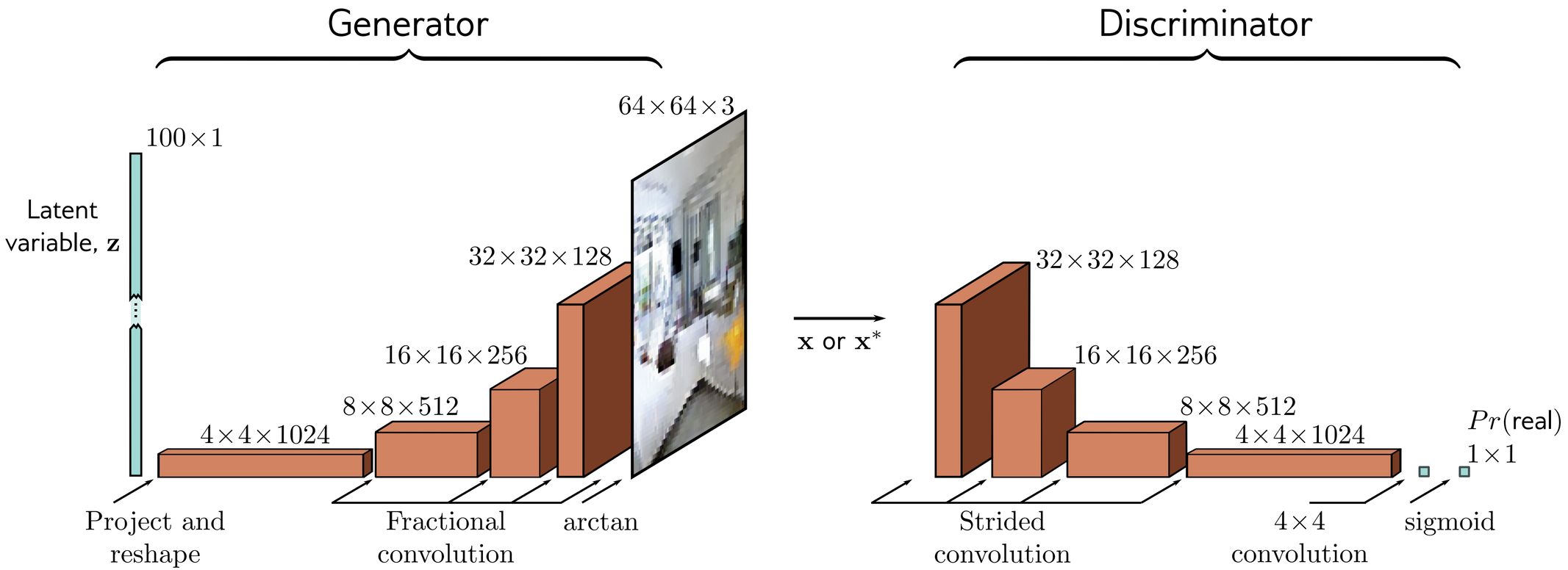
$$\sum_j \log\left[1-\text{sig}[\text{f}[\mathbf{g}[\mathbf{z}_j,\boldsymbol{\theta}],\phi]]\right]$$

Generated samples $\mathbf{x}^*=\mathbf{g}[\mathbf{z},\boldsymbol{\theta}]$
should be assigned high
probability by discriminator

Converge when the discriminator cannot tell real from fake, meaning the generator has successfully learned the data distribution.

$$p_g(x)=p_{\text{data}}(x) \qquad d(x,\phi)\sim\frac{1}{2}$$

GAN


Generator
Discriminator
$64 \times 64 \times 3$
100×1
Latent variable, $\mathbf{z}$
$32 \times 32 \times 128$
$16 \times 16 \times 256$
$8 \times 8 \times 512$
$4 \times 4 \times 1024$
$\mathbf{x}$ or $\mathbf{x}^*$
$32 \times 32 \times 128$
$16 \times 16 \times 256$
$8 \times 8 \times 512$
$4 \times 4 \times 1024$
$Pr(\text{real})$
1×1
Project and reshape
Fractional convolution
arctan
Strided convolution
4×4 convolution
sigmoid

GAN training difficulties

Random sample (trained on data face)

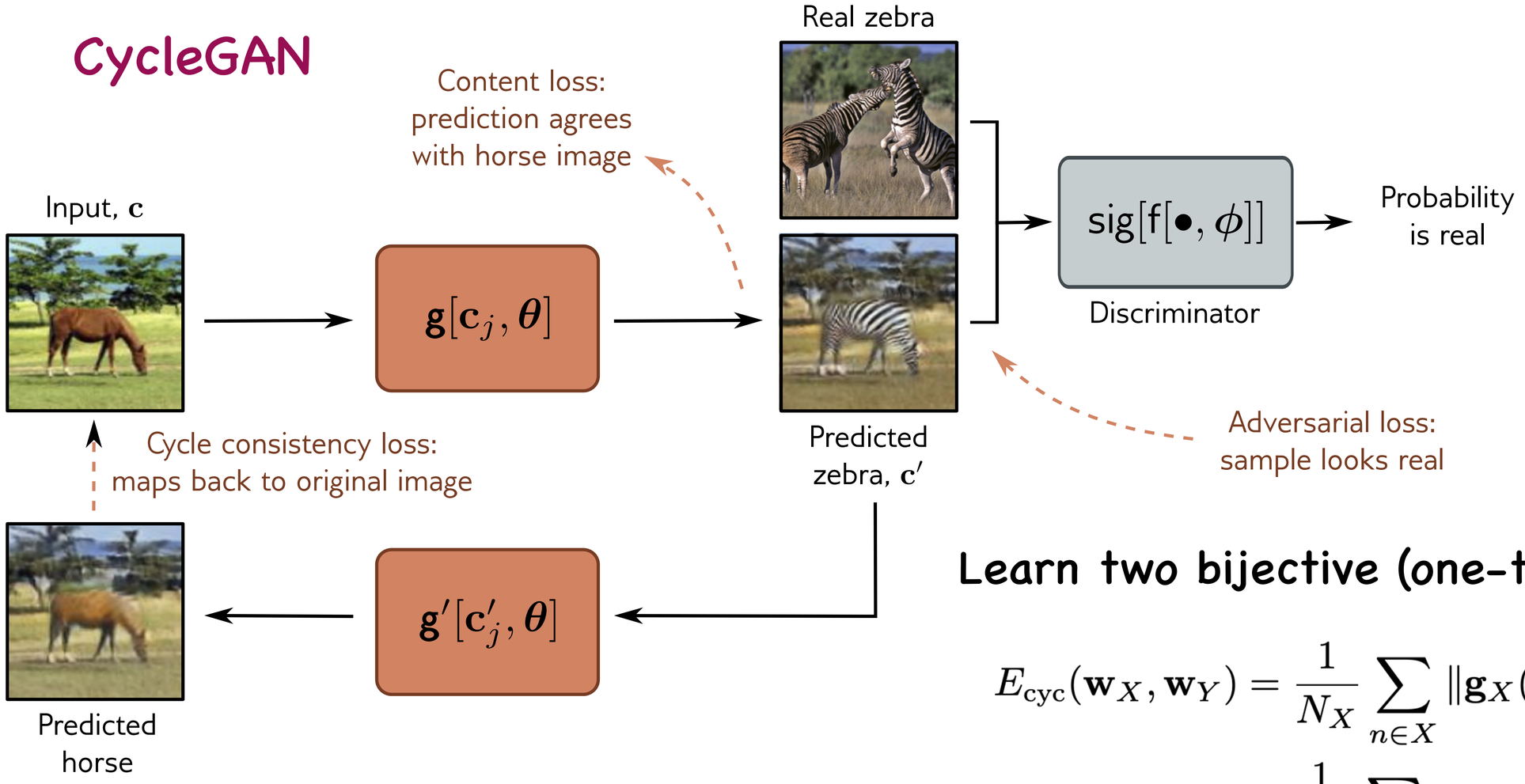
Specific training tuning

mode dropping or mode collapse

There are solutions to Improve the stability

CycleGAN

CycleGAN



Learn two bijective (one-to-one) mappings

$$E_{\text{cyc}}(\mathbf{w}_X, \mathbf{w}_Y) = \frac{1}{N_X} \sum_{n \in X} \|\mathbf{g}_X(\mathbf{g}_Y(\mathbf{x}_n)) - \mathbf{x}_n\|_1$$

$$+ \frac{1}{N_Y} \sum_{n \in Y} \|\mathbf{g}_Y(\mathbf{g}_X(\mathbf{y}_n)) - \mathbf{y}_n\|_1$$

Representation Learning

Random samples from latent space are propagated through the trained network

The generated images become organized in ways that are semantically meaningful

Normalizing flows

GANs extend the framework of linear latent-variable models

The likelihood function is intractable or not defined

The likelihood function must be evaluated

Still ensuring that sampling from the trained model is guaranteed

Normalizing flows

GANs extend the framework of linear latent-variable models

The likelihood function is intractable or not defined

The likelihood function must be evaluated

Still ensuring that sampling from the trained model is guaranteed

Generative function must be invertible

$$g(z, w)$$

$$g^{-1} \equiv f \qquad p_x(x \mid w) = p_z(f(x, w)) \left| \det \frac{\partial f(x, w)}{\partial x} \right|$$

$$\ln p(D \mid w) = \sum_i \left\{ \ln p_z(f(x_i, w) + \ln \left| \det \, \mathrm{J}(\mathrm{x_i}) \right| \right\}$$

There is a flow from nornmolize latent distribution to complex data space